Contents

ANP-002: Agent Integrity Protocol **1**
Table of Contents . . . 1
1. Abstract . . . 2
2. Introduction and Problem Statement . . . 2
2.1 Motivation . . . 2
2.2 Objectives . . . 3
3. Terminology . . . 3
4. Agent Integrity Model . . . 4
4.1 Core States (extended from ANP-001) . . . 4
4.2 Simplified Transition Diagram . . . 4
5. Integration with ANP-001 Stasis Machine . . . 4
6. Zero-Knowledge Proof Mechanism . . . 5
6.1 Chosen Primitives . . . 5
6.2 Proof Statement . . . 5
6.3 Proof Generation Flow . . . 5
7. Deterministic Validation and Interdiction . . . 5
8. Reference Implementation Considerations . . . 6
9. Security Considerations . . . 6
10. Privacy Considerations . . . 6
11. Future Work and Extensions . . . 7
12. References . . . 7

ANP-002: Agent Integrity Protocol

ArquivoNulo Protocol Series
Status: Experimental / Draft Proposal
Version: 0.1-alpha
Author: Eduardo Monteiro
Organization: ArquivoNulo Global
Date: March 2026
License: MIT

Table of Contents

1. Abstract

2. Introduction and Problem Statement

3. Terminology

4. Agent Integrity Model
5. Integration with ANP-001 Stasis Machine
6. Zero-Knowledge Proof Mechanism
7. Deterministic Validation and Interdiction
8. Reference Implementation Considerations
9. Security Considerations
10. Privacy Considerations
11. Future Work and Extensions
12. References

---

1. Abstract

ANP-002 defines a protocol for **verifiable integrity of autonomous AI agents** in distributed and high-stakes environments. It extends the Deterministic State Stasis Protocol (ANP-001) by incorporating **zero-knowledge proofs (ZKPs)** to allow agents to cryptographically demonstrate compliance with declared policies, ethical boundaries, and correct state transitions — without disclosing sensitive inputs, model parameters, reasoning traces or proprietary computations.

The protocol enables trustless auditing and coordination among agents, operators, regulators and counterparties, addressing the growing need for **provable alignment** in autonomous systems while preserving privacy and scalability.

---

2. Introduction and Problem Statement

2.1 Motivation

Autonomous AI agents (LLM-based, reinforcement learning, hybrid rule+ML) are increasingly deployed in critical domains: decentralized finance, healthcare decision support, robotic coordination, edge intelligence and multi-agent simulations. These agents operate with significant autonomy, yet their internal decision processes remain largely opaque.

Key challenges include:

- Inability to audit decisions for compliance without full disclosure

- Risk of drift, hallucination or adversarial manipulation
- Privacy conflicts when sharing logs or traces for verification
- Lack of scalable, cryptographic mechanisms for runtime and post-facto assurance

ANP-002 solves these by combining deterministic state anchoring (from ANP-001) with succinct, non-interactive zero-knowledge proofs.

2.2 Objectives

- Enable agents to **prove** policy compliance and correct execution deterministically
- Preserve confidentiality of proprietary models and sensitive data
- Provide automatic interdiction on proof failure
- Remain compatible and layered on top of ANP-001

---

3. Terminology

- **Autonomous AI Agent** — Software entity that perceives environment, reasons and acts independently
- **Policy Circuit** — Arithmetic representation of behavioral rules (ethical, legal, safety constraints)
- **Zero-Knowledge Proof (ZKP)** — Cryptographic proof of statement validity without revealing witness
- **Public Input** — Values visible to verifier (e.g. policy hash, output commitment)
- **Private Witness** — Hidden values (e.g. full input, model state, reasoning path)
- **Interdiction** — Automatic isolation or halt on verification failure

---

4. Agent Integrity Model

The integrity model extends ANP-001's six-state machine with proof generation and verification steps.

4.1 Core States (extended from ANP-001)

- S0 – Idle
- S1 – Ingestion / Perception
- S2 – Anchoring (Sovereign Hash)
- S3 – Integrity Check (drift detection)
- S4 – Stasis Lock (immutability)
- S5 – Execution / Transmission
- **S6 – Validation + Proof Generation** (new): Generate ZKP of compliance
- **S7 – Proof Verification** (external or on-chain): Verifier confirms proof

4.2 Simplified Transition Diagram

S0 → S1 (Perceive) → S2 (Anchor) → S3 (Check) → S4 (Stasis)
↓ OK
S5 (Execute)
↓
S6 (Generate ZKP)
↓
S7 (Verify Proof)
↓
Valid → SUCCESS
Invalid → INTERDICTION

---

5. Integration with ANP-001 Stasis Machine

ANP-002 **REQUIRES** ANP-001 as the base layer.

- All state transitions MUST pass through ANP-001 stasis before proof generation
- The sovereign hash from S2 MUST be used as public input to the ZKP circuit

- Interdiction from ANP-001 (S3 failure) SHORT-CIRCUITS proof generation

This ensures that only **deterministically anchored states** can produce valid proofs.

---

6. Zero-Knowledge Proof Mechanism

6.1 Chosen Primitives

- **Primary**: zk-SNARKs (Halo2, PLONK-based) — compact proofs (~200–500 bytes), fast verification
- **Future-proof alternative**: zk-STARKs (transparent, post-quantum)
- **Circuit framework**: Halo2 or Circom-style arithmetization

6.2 Proof Statement

The agent proves:

> "Given public input H = hash(policy || input_commitment || output),
> there exists a private witness W (full computation trace) such that:
> - W complies with policy circuit P
> - W starts from anchored state H_ANP001 (from ANP-001 S2)
> - output matches declared action"

6.3 Proof Generation Flow

1. Compile policy + decision logic into arithmetic circuit
2. Commit to inputs/outputs (Merkle or Pedersen)
3. Run zk-prover → produce succinct proof π
4. Output (π, public inputs) for verification

Verification time: O(1) — milliseconds even for complex LLM calls.

---

7. Deterministic Validation and Interdiction

- **MUST**: Agent generates proof π after every critical decision
- **MUST**: Submit π + public inputs to verifier (peer, smart contract, orchestrator)

- **MUST**: On invalid proof → trigger interdiction (halt agent, isolate node, log violation)
- **SHOULD**: Anchor valid proofs to immutable ledger (future ANP-003)

---

8. Reference Implementation Considerations

- Proof-of-Concept MAY extend existing stasis machine implementations
- ZKP libraries such as Halo2 or Circom SHOULD be used
- Simulations SHOULD model policy-bound decisions (e.g., regulated approvals)
- Benchmark targets: <100ms verification for simple circuits

---

9. Security Considerations

- **Soundness** — Relies on cryptographic assumptions (discrete log, knowledge of exponent)
- **Trusted setup** — Avoided in STARKs; Halo2 uses per-circuit ceremony
- **Side-channel risks** — Constant-time implementations REQUIRED
- **Denial-of-service** — Proof generation is computationally expensive → rate limiting REQUIRED
- **Quantum resistance** — Transition path to STARKs or PQ systems recommended

---

10. Privacy Considerations

- No private data revealed during verification
- Differential privacy MAY be layered for commitments
- Agents MAY rotate proving keys to prevent linkage across sessions

---

11. Future Work and Extensions

- Real PoC with Halo2 / RISC Zero
- Integration with multi-agent frameworks
- On-chain verification (future ANP-003 ledger)
- Recursive proofs for hierarchical agent systems
- Standardization path (IEEE, IETF, AI safety consortia)

---

12. References

- ANP-001: Deterministic State Stasis Protocol
- Halo2 Book — Electric Coin Company
- zk-STARKs Whitepaper — StarkWare
- NIST Post-Quantum Cryptography Standardization

Artifact Hash (SHA3-384): *To be generated after save*